- The first migration script generated for a new Django app is named 0001_initial.py, with subsequent files starting with an incremented number and the name of the class and field getting changed.
- The `Migration` class has three fields: a Boolean indicating whether this is an initial script, a list of other scripts that must be run before this one, and a list of operations to perform.
- The name of the migration script is a hint for developers regarding what the script does, but the number is irrelevant to the framework. Django determines the order of script execution based on the `depenencies` member of the `Migration` classes.
- The `operations` list in a `Migration` class contains one or more classes that indicate what changes need to happen. For example, the `CreateModel` operation indicates a new `Model` has been created and one or more tables need to be created in the database.
- The `CreateModel` operation has a `fields` member that contains descriptors of the fields in the `Model`. It uses the same classes to specify fields as a `Model` class.
- Changing an existing `Model` impacts the data in the table that is already there. You may need to provide default entries for existing rows in the database. The `makemigrations` script prompts you if this is the case and gives you access to a REPL to enter the default data to be used by the script.
- Migrations can be run both forward and backward.
- The `RunPython` operation allows you to call a function to perform side-effects on the database as part of the migration. `RunPython` takes two function references as arguments, the first to be called when migrating forward and the second for migrating backward.
- You can generate a migration script with no operations using the `makemigrations` command and passing it the `--empty` flag. This can be used to create a stub file for custom `RunPython` operations.
- Renaming or combining fields requires a three-step process: (1) add the new field and migrate the change; (2) add a custom migration script that uses the `RunPython` operation to migrate the data and migrate the change; and (3) remove the original fields and migrate the change.
- Not all custom migrations can be undone. If a migration can't be undone, the second argument to `RunPython` can be left empty. Django raises an exception if such a backward migration gets attempted.
- For efficiency, multiple migration files can be combined using the `squashmigrations` management command. This command does not remove the old files, which can be left in place, as they are no longer part of the dependency chain.